

05

The security pre-flight

Row-level security done right, the auth pattern that survives, and the hardcoded keys you left in. Two checks catch most of it.

SUBJECT	AUTHORIZATION • SECRETS
AUTHOR	MICAH JONES
TIME	A TEN-MINUTE READ

READER	SOLO BUILDERS ON AI TOOLS
STATUS	CHAPTER FIVE OF TEN
REV	2026.08

Nobody is targeting you. Everything is scanning you.

The chapter-four ritual ended with strangers able to reach your app. Here is the uncomfortable arithmetic of that: within hours of your domain going live, automated scanners found it. Not hackers who care about you. Scripts that knock on every door on the internet, all day, forever, looking for the same handful of unlocked ones.

That reframing is good news. You are not defending against genius. You are defending against a checklist, which means a checklist can defend you. Solo-builder security is not a discipline you study for a year. It is two questions, asked honestly, plus the habits from chapters one through four you already have.

Check one: can the wrong user read someone else's rows?

Check two: is anything secret reachable from public?

Most of what actually goes wrong in solo builds is one of these two, wearing different clothes.

The distinction the AI skips

AUTHENTICATION VS. AUTHORIZATION

Authentication is who you are: the login. Authorization is what you may do: which rows, which files, which buttons are truly yours. AI tools wire authentication well, because it comes from a library. Authorization they skip, because it comes from your product's rules, and chapter two told you where those live: in your head.

This is why the most common vulnerability in AI-built apps is embarrassingly plain: the login works perfectly, and then any logged-in user can read any other user's data by changing a number in the URL. The industry calls it an IDOR. You can call it the "everyone is admin" bug. The login was never the hard part.

The auth pattern that survives has three rules. Use the platform's auth, never your own: password storage is a solved problem and your version will be worse. Keep the session in the server-managed cookie the library gives you. And the load-bearing one: the server learns who is asking from the session, never from what the client sent. Any request that carries "userId" as an input is a request the AI wrote for the demo, not for strangers.

§ 05.3

Row-level security, done right

Chapter three called ownership-in-the-database "the rule is physics." Here is the whole recipe, concretely. It is shorter than most readers expect.

Every table gets an owner column. Row-level security gets switched on for every table, which flips the database to deny-by-default: a query with no matching policy returns nothing. Then one policy per access pattern, and for a solo SaaS there is usually just one that matters:

migration: lock clients to their trainer

```
alter table clients enable row level security;
create policy "trainers see only their own clients"
on clients for all
using (trainer_id = auth.uid());
-- auth.uid() = the signed-in user. The
-- helper's name varies by database host.
```

Ten lines like these, and the "everyone is admin" bug becomes structurally impossible: a forgetful query returns an empty list instead of the whole customer base. The AI can rewrite your queries badly forever and the database keeps the rule, because the rule no longer lives in the queries.

Two keys come with this setup, and they are opposites. The anonymous key ships to the browser and is safe to be public, precisely because the policies stand behind it. The service key bypasses every policy and belongs only in the server's environment variables, chapter four style. Mixing those two up is the one way to undo everything above.

And the test, because chapter one taught you that rules you don't check are rules you don't have: **the two-account test**.

FIELD NOTE

Ordani, the HIPAA-compliant app from these pages, runs its entire authorization model this way: a birth worker sees her clients and nobody else's, enforced in the database. Both outside security reviews walked those policies line by line.

Create data as user A. Log in as user B and try to read it: through the app, and through the lazy door, by editing IDs in the URL and in API calls. Five minutes. If B sees anything of A's, you have tonight's work, and you found it before a stranger's script did.

The keys you left in

§ 05.4

Check two is about secrets, and there are exactly three places they hide in an AI-built repo.

In the code and config. Chapter three's build-log entry found one live token pasted twelve times by helpful sessions. The grep-for-credential-shapes habit covers this, if you actually run it.

In the browser bundle. Framework variables with a public prefix ship to every visitor. The test is view-source honesty: open the deployed site, view the page source and the JavaScript it loads, and search for `sk_`, `secret`, `key`. If a privileged key appears, every visitor already has it.

In the history. Git remembers. A key committed once and deleted later is still in the repository's past, and anything that has ever synced to another machine has copies. Which is why the rule for a leaked key is never "delete it." It is rotate it: issue a new one, kill the old one, and the history can keep its souvenir.

The secret I leaked to be helpful

This one is mine, from this week. Moving fast on my own site, I pasted two live credentials, an email API key and a private calendar URL, straight into an AI chat session to get a feature wired. It worked. The feature shipped that morning.

And the moment they hit the chat, both stopped being secrets. Not because anyone malicious was watching, but because they had touched a surface I don't control, and "probably fine" is not a security model. Rotation went on that day's list, with a deadline, written where I can't ignore it, before the feature was even finished.

If reading this just reminded you of a key you pasted somewhere once, congratulations: you have found your rotation list. The question is never "did it leak?" It is "can I afford to assume it didn't?"

Pre-flight: the five checks

PRE-FLIGHT • SECURITY

RUN TONIGHT

☐ **Run the two-account test.** Create as A, snoop as B, through the app and through edited IDs. Anything visible is tonight's work.

☐ **Row-level security on, on every table.** Deny by default, one policy per pattern, service key server-side only.

☐ **View-source the deployed site.** Search the page and its scripts for anything key-shaped. The browser bundle is public forever.

☐ **Grep the repo and its history for credential shapes.** Anything found gets rotated, never just deleted.

☐ **Hit your admin routes logged out.** Cold, in an incognito window. Hidden buttons are not authorization, and scanners don't use buttons.

Two questions, five checks, one evening. That is most of solo-builder security, and it is more than the scanners are betting you did. The next chapter follows the money: the payment stack, where the stakes stop being embarrassment and start being refunds.

NEXT • CHAPTER 06

Stripe in production

Webhook reliability, refunds, subscription edge cases, and the test-to-live failures nobody warns you about.